

1 Loading the dataset

```
[2]: import pandas as pd
```

```
df = pd.read_csv("california_housing.csv")
df
```

```
[2]:
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY
...	...	...	...	...	...	...	...	...	...	...
20635	-121.09	39.48	25.0	1665.0	374.0	845.0	330.0	1.5603	78100.0	INLAND
20636	-121.21	39.49	18.0	697.0	150.0	356.0	114.0	2.5568	77100.0	INLAND
20637	-121.22	39.43	17.0	2254.0	485.0	1007.0	433.0	1.7000	92300.0	INLAND
20638	-121.32	39.43	18.0	1860.0	409.0	741.0	349.0	1.8672	84700.0	INLAND
20639	-121.24	39.37	16.0	2785.0	616.0	1387.0	530.0	2.3886	89400.0	INLAND

20640 rows × 10 columns

2 Performing EDA

```
[3]: df.head()
```

```
[3]:
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY

```
[4]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column              Non-Null Count  Dtype  
---  -
0   longitude            20640 non-null float64
1   latitude             20640 non-null float64
2   housing_median_age   20640 non-null float64
3   total_rooms          20640 non-null float64
4   total_bedrooms       20433 non-null float64
5   population            20640 non-null float64
6   households            20640 non-null float64
7   median_income         20640 non-null float64
8   median_house_value    20640 non-null float64
9   ocean_proximity       20640 non-null str
dtypes: float64(9), str(1)
memory usage: 1.6 MB
```

```
[5]: df.describe()
```

```
[5]:
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000	20640.000000	20640.000000	20640.000000	20640.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553	1425.476744	499.539680	3.870671	206855.816909
std	2.003532	2.135952	12.585558	2181.615252	421.385070	1132.462122	382.329753	1.899822	115395.615874
min	-124.350000	32.540000	1.000000	2.000000	1.000000	3.000000	1.000000	0.499900	14999.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000	787.000000	280.000000	2.563400	119600.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	1166.000000	409.000000	3.534800	179700.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	1725.000000	605.000000	4.743250	264725.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	35682.000000	6082.000000	15.000100	500001.000000

```
[6]: df.shape
```

```
[6]: (20640, 10)
```

Separate X and y

Now that we know what we're trying to predict, we can actually perform the ML operation.

```
[7]: X = df.drop(columns = "median_house_value")
     y = df["median_house_value"]
```

[8]: X

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	ocean_proximity
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	NEAR BAY
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	NEAR BAY
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	NEAR BAY
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	NEAR BAY
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	NEAR BAY
...	...	...	...	...	...	...	...	...	...
20635	-121.09	39.48	25.0	1665.0	374.0	845.0	330.0	1.5603	INLAND
20636	-121.21	39.49	18.0	697.0	150.0	356.0	114.0	2.5568	INLAND
20637	-121.22	39.43	17.0	2254.0	485.0	1007.0	433.0	1.7000	INLAND
20638	-121.32	39.43	18.0	1860.0	409.0	741.0	349.0	1.8672	INLAND
20639	-121.24	39.37	16.0	2785.0	616.0	1387.0	530.0	2.3886	INLAND

20640 rows × 9 columns

```
[9]: y # Target variable (y) = median_house_value
```

```
[9]: 0      452600.0
     1      358500.0
     2      352100.0
     3      341300.0
     4      342200.0
     ...
    20635      78100.0
    20636      77100.0
    20637      92300.0
    20638      84700.0
    20639      89400.0
     Name: median_house_value, Length: 20640, dtype: float64
```

Verify

```
[10]: print("X shape:", X.shape)
      print("y shape:", y.shape)
```

X shape: (20640, 9)
y shape: (20640,)

Train/Test Split

Objective

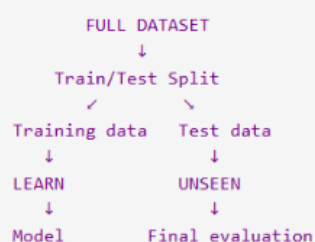
We want to train a model that learns the relationship between:

Housing features → Median house value

But there's a problem:

If we train and evaluate the model on the same data, we can't properly know whether it can generalize to new houses.

So we divide the data into:



```
[11]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size = 0.2,
    random_state = 42
)
```

```
[12]: print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (16512, 9)
X_test shape: (4128, 9)
y_train shape: (16512,)
y_test shape: (4128,)
```

Check the training data

```
[13]: X_train.isnull().sum()
```

```
[13]: longitude          0
latitude              0
housing_median_age    0
total_rooms           0
total_bedrooms        0
population            0
households            0
median_income         0
ocean_proximity       0
dtype: int64
```

```
[14]: X_test.isnull().sum()
```

```
[14]: longitude          0
latitude              0
housing_median_age    0
total_rooms           0
total_bedrooms        207
population            0
households            0
median_income         0
ocean_proximity       0
dtype: int64
```

```
[15]: # Create the imputer
```

```
[17]: from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy = "median")

imputer.fit(X_train[["total_bedrooms"]])
X_train["total_bedrooms"] = imputer.transform(X_train[["total_bedrooms"]])
```

```
[18]: # Transform X_test

[19]: X_test["total_bedrooms"] = imputer.transform(
      X_test[["total_bedrooms"]]
      )

[20]: # verify

[21]: X_train.isnull().sum()

[21]: longitude      0
      latitude      0
      housing_median_age  0
      total_rooms    0
      total_bedrooms  0
      population     0
      households     0
      median_income  0
      ocean_proximity  0
      dtype: int64

[22]: X_test.isnull().sum()

[22]: longitude      0
      latitude      0
      housing_median_age  0
      total_rooms    0
      total_bedrooms  0
      population     0
      households     0
      median_income  0
      ocean_proximity  0
      dtype: int64
```

▼ Categorical Encoding

Our California Housing dataset has a categorical feature:

ocean_proximity

It contains text categories such as:

<1H OCEAN INLAND NEAR OCEAN NEAR BAY ISLAND

A Linear Regression model cannot directly calculate with strings like "INLAND".

So we need to encode the categories into numbers.

🔗 Step 1 — See our actual categories

```
[23]: X_train["ocean_proximity"].value_counts()

[23]: ocean_proximity
      <1H OCEAN    7341
      INLAND      5227
      NEAR OCEAN   2086
      NEAR BAY    1854
      ISLAND        4
      Name: count, dtype: int64

[24]: # One-Hot Encoding
```



Scaling numerical values

```
[33]: from sklearn.preprocessing import StandardScaler
      scaler = StandardScaler()
      numeric_cols = X_train.select_dtypes(include="number").columns
      scaler.fit(X_train[numeric_cols])
      X_train_scaled = scaler.transform(
          X_train[numeric_cols]
      )

[34]: X_test_scaled = scaler.transform(
      X_test[numeric_cols]
      )

[35]: # Check the result

[38]: X_train_scaled.mean(axis=0)

[38]: array([ 1.77678716e-15,  3.38574991e-15, -1.67824411e-17,  3.52861581e-17,
        -4.47531762e-17,  0.00000000e+00, -5.50808322e-17, -6.41175313e-17])

[39]: X_train_scaled.std(axis=0)

[39]: array([1., 1., 1., 1., 1., 1., 1., 1.])

[40]: # outlier detection
      # Choose a numerical feature
```

outlier detection

Choose a numerical feature

```
[42]: X_train["total_rooms"].describe()

[42]: count    16512.000000
      mean     2642.004784
      std     2174.646744
      min        2.000000
      25%     1454.000000
      50%     2129.000000
      75%     3160.000000
      max    39320.000000
      Name: total_rooms, dtype: float64

[43]: Q1 = X_train["total_rooms"].quantile(0.25)
      Q3 = X_train["total_rooms"].quantile(0.75)

      IQR = Q3 - Q1

      print("Q1:", Q1)
      print("Q3:", Q3)
      print("IQR:", IQR)

      Q1: 1454.0
      Q3: 3160.0
      IQR: 1706.0
```

```
[44]: lower = Q1 - 1.5 * IQR
      upper = Q3 + 1.5 * IQR

      print("Lower boundary:", lower)
      print("Upper boundary:", upper)
```

```
Lower boundary: -1105.0
Upper boundary: 5719.0
```

```
[46]: outliers = X_train[X_train["total_rooms"] > upper
      ]
      print("Number of potential outlier:", len(outliers))
```

```
Number of potential outlier: 1046
```

```
[47]: outliers["total_rooms"].describe()
```

```
[47]: count    1046.000000
      mean     8755.678776
      std     3890.511003
      min     5721.000000
      25%     6339.000000
      50%     7362.000000
      75%     9613.750000
      max     39320.000000
      Name: total_rooms, dtype: float64
```

```
[49]: numeric_cols = X_train.select_dtypes(include="number").columns

      outlier_summary = []

      for col in numeric_cols:
          Q1 = X_train[col].quantile(0.25)
          Q3 = X_train[col].quantile(0.75)
          IQR = Q3 - Q1

          lower = Q1 - 1.5 * IQR
          upper = Q3 + 1.5 * IQR

          count = ((X_train[col] < lower) | (X_train[col] > upper)).sum()

          outlier_summary.append([col, count, lower, upper])

      outlier_summary = pd.DataFrame(
          outlier_summary,
          columns=["Feature", "Outlier_Count", "Lower_Bound", "Upper_Bound"]
      )

      outlier_summary
```

```
[49]:
```

	Feature	Outlier_Count	Lower_Bound	Upper_Bound
0	longitude	0	-127.510000	-112.310000
1	latitude	0	28.245000	43.405000
2	housing_median_age	0	-10.500000	65.500000
3	total_rooms	1046	-1105.000000	5719.000000
4	total_bedrooms	1041	-228.625000	1172.375000
5	population	955	-616.500000	3131.500000
6	households	977	-209.000000	1095.000000
7	median_income	529	-0.743013	8.082888

```
[50]: X_train["total_bedrooms"].describe()
```

```
[50]: count    16512.000000
      mean      538.496851
      std       419.007096
      min        1.000000
      25%       296.750000
      50%       437.000000
      75%       647.000000
      max      6445.000000
      Name: total_bedrooms, dtype: float64
```

```
[51]: X_train["median_income"].describe()
```

```
[51]: count    16512.000000
      mean      3.880754
      std       1.904294
      min       0.499900
      25%       2.566700
      50%       3.545800
      75%       4.773175
      max      15.000100
      Name: median_income, dtype: float64
```

Feature Engineering

```
[54]: # Create rooms_per_household
```

```
[57]: X_train["rooms_per_household"] = (
      X_train["total_rooms"] / X_train["households"]
      )
      X_test["rooms_per_household"] = (
      X_test["total_rooms"] / X_test["households"]
      )
```

```
[60]: # bedrooms_per_room
      X_train["bedrooms_per_room"] = (
      X_train["total_bedrooms"] / X_train["total_rooms"]
      )
      X_test["bedrooms_per_room"] = (
      X_test["total_bedrooms"] / X_test["total_rooms"]
      )
```

```
[61]: X_train["population_per_household"] = (
      X_train["population"] / X_train["households"]
      )
      X_test["population_per_household"] = (
      X_test["population"] / X_test["households"]
      )
```

```
[62]: X_train[
      [
          "rooms_per_household",
          "bedrooms_per_room",
          "population_per_household"
      ]
    ].head()
```

```
[62]:
```

	rooms_per_household	bedrooms_per_room	population_per_household
14196	5.017657	0.200576	3.691814
8267	4.473545	0.232703	1.738095
17445	5.645833	0.174486	2.723214
14265	4.002817	0.258269	3.994366
2271	6.268421	0.180940	2.300000

```
[63]: # Look at correlations with our target
```

```
[64]: y_train
```

```
[64]: 14196    103000.0
      8267    382100.0
      17445   172600.0
      14265    93400.0
      2271    96500.0
      ...
      11284   229200.0
      11964    97800.0
      5390    222100.0
      860     283500.0
      15795   325000.0
      Name: median_house_value, Length: 16512, dtype: float64
```

```
[66]: train_analysis = X_train.copy()
      train_analysis["median_house_value"] = y_train
```

```
[67]: #Calculate correlations
```

```
[68]: correlations = train_analysis.corr(numeric_only=True)["median_house_value"]
      correlations.sort_values(ascending=False)
```

```
[68]: median_house_value    1.000000
      median_income       0.690647
      rooms_per_household  0.158485
      total_rooms         0.133989
      housing_median_age   0.103706
      households          0.063714
      total_bedrooms       0.047980
      population_per_household -0.022030
      population          -0.026032
      longitude           -0.046349
      latitude            -0.142983
      bedrooms_per_room    -0.257419
      Name: median_house_value, dtype: float64
```

```
[69]: feature_corr = X_train.corr(numeric_only=True)
feature_corr
```

```
[69]:
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	rooms_per_household	be
longitude	1.000000	-0.924485	-0.101818	0.038676	0.063064	0.094276	0.049306	-0.017040	-0.029339	
latitude	-0.924485	1.000000	0.005296	-0.029224	-0.059998	-0.102499	-0.064061	-0.076571	0.110695	
housing_median_age	-0.101818	0.005296	1.000000	-0.360922	-0.320624	-0.292283	-0.302796	-0.121711	-0.160892	
total_rooms	0.038676	-0.029224	-0.360922	1.000000	0.930489	0.857936	0.920482	0.198268	0.136090	
total_bedrooms	0.063064	-0.059998	-0.320624	0.930489	1.000000	0.878932	0.980255	-0.009141	-0.001659	
population	0.094276	-0.102499	-0.292283	0.857936	0.878932	1.000000	0.907452	0.004122	-0.073733	
households	0.049306	-0.064061	-0.302796	0.920482	0.980255	0.907452	1.000000	0.012776	-0.083062	
median_income	-0.017040	-0.076571	-0.121711	0.198268	-0.009141	0.004122	0.012776	1.000000	0.336013	
rooms_per_household	-0.029339	0.110695	-0.160892	0.136090	-0.001659	-0.073733	-0.083062	0.336013	1.000000	
bedrooms_per_room	0.097280	-0.118938	0.135495	-0.189316	0.084149	0.035134	0.064185	-0.616669	-0.435169	
population_per_household	-0.000598	0.005837	0.016245	-0.024991	-0.028536	0.072330	-0.027656	0.022061	-0.004922	

Build the Preprocessing Pipeline

"Whenever new data arrives, perform these preprocessing steps in this exact order."

This prevents mistakes and makes the workflow reproducible.

```
[72]: # Step 1 - Separate numerical and categorical columns
```

```
[73]: categorical_cols = ["ocean_proximity"]
```

```
[74]: # And we can automatically identify the numerical columns:
```

```
[89]: numeric_cols = X_train.select_dtypes(include="number").columns.tolist()
```

```
[90]: # Step 2 - Import the tools
```

```
[91]: from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```
[92]: # Step 3 - Build the numerical preprocessing pipeline
```

```
[93]: numeric_pipeline = Pipeline([("imputer", SimpleImputer(strategy="median")),("scaler", StandardScaler())])
```

```
[94]: # Step 4 - Build the categorical preprocessing pipeline
```

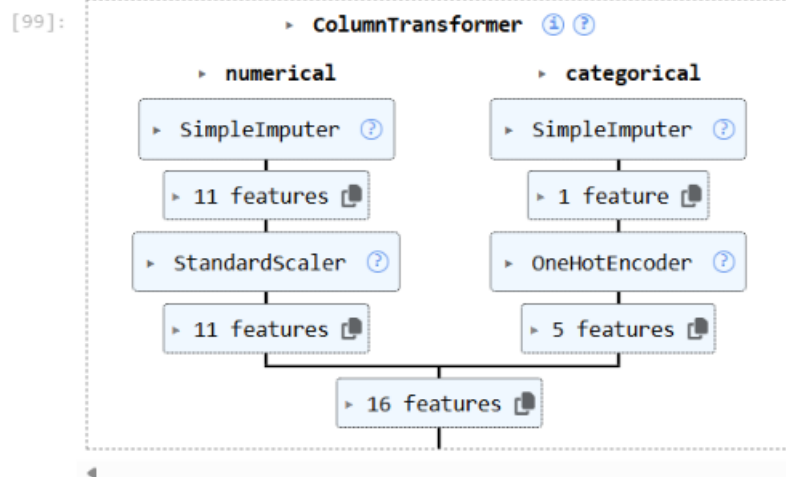
```
[95]: categorical_pipeline = Pipeline([("imputer", SimpleImputer(strategy="most_frequent")),
                                     ("encoder", OneHotEncoder(handle_unknown="ignore"))
                                     ])
```

```
[96]: # Step 5 - Combine them with ColumnTransformer
```

```
[97]: preprocessor = ColumnTransformer([("numerical", numeric_pipeline, numeric_cols),  
                                     ("categorical", categorical_pipeline, categorical_cols)  
                                     ])
```

```
[98]: # Step 6 - Fit the preprocessor
```

```
[99]: preprocessor.fit(X_train)
```



```
[100]: X_train_processed = preprocessor.transform(X_train)
```

```
[101]: X_test_processed = preprocessor.transform(X_test)
```

```
[102]: # Step 7 Check the result
```

```
[103]: X_train_processed.shape  
X_test_processed.shape
```

```
[103]: (4128, 16)
```

```
[104]: print("Training data:", X_train_processed.shape)  
print("Test data:", X_test_processed.shape)
```

```
Training data: (16512, 16)
```

```
Test data: (4128, 16)
```